Modèles algorithmiques fondamentaux sur structures à accès direct

Thème principal : Initiation à l'algorithmique et patrons de conception (Design Patterns) sur structures indicées

Synthèse exécutive : Ce cours pose les fondements méthodologiques indispensables de l'algorithmique en BUT 1 Informatique. Avant d'écrire la moindre ligne de code, l'informaticien doit analyser le contrat d'interface et le mécanisme d'accès de sa structure de données (Direct, Séquentiel ou Dynamique). Le cours formalise trois grands modèles algorithmiques canoniques applicables aux structures à accès direct (tableaux, chaînes de caractères) : (1) le parcours complet avec traitement systématique, (2) le parcours complet avec traitement conditionné, et (3) la recherche de première occurrence. Sont également détaillés : la gestion rigoureuse des bornes d'indices (de 0 à N-1 pour éviter les erreurs de dépassement 'out of bounds'), le principe du pessimisme algorithmique, l'obligation incontournable d'une double condition d'arrêt pour toute recherche, ainsi que le comparatif précis entre les deux variantes d'implémentation d'une recherche selon que l'on vise la simplicité syntaxique ou la conservation de la position indicée exacte.

1. Le Pourquoi & Les Mécanismes d'Accès aux Structures de Données

L'étape zéro de toute conception algorithmique consiste à analyser le contrat d'interface offert par la structure de données sous-jacente. Si la structure propose un accès direct (comme un tableau statique ou une chaîne de caractères), chaque case mémoire est immédiatement adressable par son indice. En revanche, un flux séquentiel (tel qu'un fichier texte brut ou une liste chaînée simple) n'expose que trois opérations primitives : se positionner au début, avancer d'un pas vers l'élément suivant, et tester si la fin de flux est atteinte. Tenter d'utiliser une boucle itérative indicée de type 'T[i]' sur un flux séquentiel constitue une erreur conceptuelle rédhibitoire qui invalide totalement l'algorithme.

- Structure de données : Collection regroupant un ensemble d'éléments de même type (tableaux, chaînes de caractères).
- Accès direct : Mécanisme permettant d'atteindre instantanément n'importe quel élément via son indice ou sa clé, sans parcourir les précédents (Complexité O(1)).
- Accès séquentiel : Contrainte imposant de se positionner au début puis de progresser pas à pas d'un élément au suivant (début, suivant, fin).
- Accès dynamique : Structure hybride proposant à la fois des primitives d'accès séquentiel et d'accès direct.
- Incompatibilité structurelle : Interdiction absolue d'appliquer un modèle nécessitant l'accès direct (ex: T[i]) sur un flux purement séquentiel (ex: fichier CSV brut).

Cas concret enseignant : *Le professeur met en garde contre l'erreur classique observée en contrôle : tenter d'écrire 'T[i]' sur un fichier CSV ou un fichier texte ouvert. Comme le système d'exploitation ne fournit qu'un curseur séquentiel ligne à ligne, un tel code est sémantiquement faux et conduit directement à un zéro sur la question.*

2. Premier Modèle : Le Parcours Complet avec Traitement Systématique

Ce modèle algorithmique s'applique dès lors qu'aucune exclusion n'est permise entre deux bornes définies. Il répond à des problématiques telles que : afficher toutes les valeurs d'un tableau, initialiser chaque case à zéro, ou convertir chaque caractère d'une chaîne en majuscule. En pseudo-code normalisé, pour un tableau de taille N, l'indice i démarre à 0 et termine impérativement à N - 1. Le traitement est appliqué aveuglément à chaque tour de boucle sans test préalable.

- Objectif : Appliquer un traitement uniforme à 100% des éléments situés entre une borne de début et une borne de fin.
- Structure itérative : Utilisation obligatoire d'une boucle bornée 'Pour' (car le nombre d'itérations est calculable à l'avance : Fin - Début + 1).

• Gestion stricte des bornes : Pour une structure de N éléments indicée à partir de 0, la boucle varie strictement de 0 à N - 1.

• Risque de débordement : Boucler jusqu'à N engendre une exception critique de dépassement d'indice (out of bounds).

Cas concret enseignant : *Exemple concret donné en amphi : 'Afficher un tableau de N cases'. L'enseignant rappelle que l'erreur la plus fréquente des débutants est d'écrire 'Pour i allant de 0 à N', ce qui provoque un crash immédiat à l'exécution lors de la tentative de lecture de la case mémoire T[N] qui n'appartient pas au tableau.*

3. Second Modèle : Le Parcours Complet avec Traitement Conditionnel

Le modèle du parcours complet conditionné conserve l'inspection systématique de l'ensemble des éléments, mais protège l'exécution de l'action de traitement par une condition logique. Il est utilisé pour compter des occurrences particulières (ex: compter les voyelles dans un texte), filtrer des données (ex: afficher uniquement les entiers pairs) ou accumuler des valeurs spécifiques. Si la condition implique des éléments adjacents (ex: T[i] > T[i-1]), le concepteur doit impérativement réajuster les bornes de la boucle (ex: démarrer à 1 au lieu de 0) pour empêcher tout déréférencement hors limites.

• Parcours exhaustif : Tous les éléments de la zone sont inspectés de la borne de début à la borne de fin.

• Traitement sélectif : L'action n'est déclenchée que si l'élément inspecté valide un prédicat logique (test booléen).

• Architecture : Imbrication d'une structure alternative 'Si ... Alors' au sein d'une boucle bornée 'Pour'.

• Conditions complexes : Le prédicat peut combiner plusieurs critères logiques (ET, OU) ou comparer l'élément courant avec ses voisins immédiats (i-1 ou i+1).

Cas concret enseignant : *L'analogie de la chaîne de montage en usine : 'Chaque pièce défile obligatoirement devant le contrôleur qualité (parcours complet systématique d'inspection), mais le contrôleur n'appose le tampon de rebut (action de traitement) que si et seulement si la pièce présente une fissure (traitement conditionné).'*

4. Troisième Modèle : La Recherche de Première Occurrence & Ses Variantes

Contrairement aux deux premiers modèles, la recherche de première occurrence stoppe l'itération dès que la cible est identifiée. L'informaticien doit adopter une démarche pessimiste en postulant que l'élément n'existe pas. Un algorithme de recherche valide DOIT posséder deux conditions d'arrêt : (1) le succès de la recherche, (2) l'épuisement complet de la collection. Deux variantes structurelles existent : la variante 1 (rupture immédiate, idéale pour récupérer la position exacte) et la variante 2 (Tant que global, plus simple à structurer syntaxiquement).

• Pessimisme algorithmique : Initialisation systématique d'un indicateur booléen 'trouvé' à Faux avant d'entamer l'exploration.

• Itération non bornée : Utilisation obligatoire d'une boucle 'Tant que' car le nombre de répétitions est inconnu a priori.

• Double condition d'arrêt obligatoire : Arrêt dès que l'élément est trouvé OU dès que la structure a été entièrement explorée sans succès.

• Variante 1 (Rupture immédiate) : Interruption nette dès la découverte ; conserve l'indice i exactement positionné sur l'élément trouvé.

• Variante 2 (Tant que global) : Test unique en tête ('Tant que i < N et NON trouvé') ; syntaxiquement plus standard mais avance d'un pas supplémentaire avant de sortir.

Cas concret enseignant : *Devise du prof : 'En informatique, soyez pessimistes ! Partez toujours du principe que ce que vous cherchez n'existe pas dans votre collection, et ne basculez votre indicateur à Vrai qu'au moment précis où vous le touchez.' Il martèle que tout algorithme de recherche présenté en examen avec une seule condition d'arrêt est sanctionné par une note éliminatoire.*

■ Questions & Interventions des Étudiants en Classe

Question étudiant : Est-ce qu'on peut appliquer le parcours complet avec traitement systématique sur une structure à accès dynamique ?
Réponse du professeur : Oui, tout à fait. Une structure à accès dynamique proposant à la fois des primitives séquentielles et directes, il est tout à fait possible d'exploiter ses mécanismes d'accès direct pour appliquer ce modèle.

Question étudiant : Pourquoi s'embêter à présenter deux variantes différentes pour la recherche de première occurrence ?
Réponse du professeur : Parce que le choix dépend du besoin : la variante 1 (avec rupture) est indispensable si l'on a besoin de récupérer la position exacte (l'indice) de l'élément trouvé. La variante 2 (avec Tant que global) offre un code plus épuré et plus simple à structurer syntaxiquement lorsqu'on cherche seulement une simple existence booléenne.

Question étudiant : Comment fait-on si la condition de recherche dans un parcours conditionnel dépend de l'élément précédent ou suivant ?
Réponse du professeur : Il faut simplement adapter les bornes de la boucle 'Pour' pour éviter tout dépassement d'indice (par exemple, commencer à l'indice 1 si l'on compare T[i] avec T[i-1]), mais la structure fondamentale du parcours conditionné reste strictement identique.

Question étudiant : Si l'on veut récupérer TOUS les indices correspondant à une valeur (par exemple trouver tous les '5' d'un tableau), quel modèle doit-on utiliser ?
Réponse du professeur : Il ne faut pas utiliser un modèle de recherche de première occurrence (qui s'arrête au premier trouvé), mais un 'Parcours complet avec traitement conditionnel', qui inspectera l'intégralité du tableau et collectera toutes les occurrences.

■ Exercice & Démonstration d'Application

Énoncé : Écrire en pseudo-code algorithmique rigoureux l'algorithme permettant d'afficher tous les nombres strictement positifs d'un tableau d'entiers 'T' contenant 'N' éléments, en explicitant le modèle retenu.

Résolution pas à pas :
Étape 1 : Analyse du problème et sélection du modèle. Le problème exige d'inspecter l'ensemble des éléments d'un tableau (structure à accès direct) de la première à la dernière case. L'action d'affichage ne doit être exécutée que si l'élément respecte une propriété arithmétique (valeur > 0). Le modèle canonique est donc le 'Parcours complet avec traitement conditionné'. Étape 2 : Définition des bornes d'indices. Le tableau comporte N cases. En convention informatique standard à indexation zéro, la première case est à l'indice 0 et la dernière case est à l'indice N - 1. Étape 3 : Choix de la structure de contrôle. Nombre d'itérations connu à l'avance -> boucle bornée 'Pour i allant de 0 à N - 1'. Étape 4 : Définition du prédicat logique et du traitement. À chaque itération, on évalue la condition 'T[i] > 0'. Si cette assertion est vraie, on déclenche l'instruction 'Afficher(T[i])'.

Résultat : Algorithme AffichagePositifs Données T : Tableau d'entiers N : Entier (taille du tableau) Variables i : Entier Début Pour i allant de 0 à N - 1 Faire Si T[i] > 0 Alors Afficher(T[i]) Fin Si Fin Pour Fin Commentaire didactique : Cet algorithme assure l'exhaustivité du traitement sans risque de débordement mémoire (l'indice i s'arrête strictement à N-1) et filtre sélectivement les valeurs requises.

■ Auto-Évaluation Flash d'Examen

Q : Quel modèle algorithmique doit-on employer pour vérifier si un tableau d'entiers contient au moins une fois la valeur 10 ?

R : La recherche de première occurrence (itérative non bornée avec double condition d'arrêt).

Q : Pourquoi l'indice de fin d'un parcours complet sur un tableau de N éléments est-il obligatoirement N - 1 et non N ?

R : Parce que les tableaux en informatique commencent à l'indice 0 ; une collection de N éléments possède donc ses indices étalés de 0 à N - 1. Tenter d'accéder à l'indice N provoque une erreur fatale de dépassement de capacité (out

of bounds).

Q : Combien de conditions d'arrêt minimales doit comporter la structure de contrôle d'une recherche de première occurrence ?

R : Exactement deux conditions d'arrêt minimales : (1) l'élément a été trouvé, ou (2) la structure a été entièrement explorée sans succès.